Web GIS Laboratory 3

District Dashboard: layers, controls and interaction

Dr. Shahid Nawaz Khan
Institute of Geographical Information Systems
School of Civil and Environmental Engineering
National University of Sciences and Technology, Islamabad

MS Remote Sensing and GIS

Duration	3 hours
Covers	Lecture: Leaflet, building the map in the browser
Builds on	Lab 2, District Explorer
Software	QGIS, Visual Studio Code, a browser, Git
Deliverable	A published dashboard and a short report

Contents

Before you begin 2

1 Set up the project 3

2 Make a point layer in QGIS 4

3 The page, the map and its basemaps 5

4 A thematic layer, done properly 8

5 Interaction: highlight and an info panel 11

6 Proportional symbols in their own pane 13

7 Map events, on and off 16

8 Filter by province 17

9 SVG or canvas 18

10 Publish and submit 19

Appendix A: Troubleshooting 21

Appendix B: Reference 21

Before you begin

What you will build

In Lab 2 you drew one layer and changed it with one drop-down. In this lab you turn that map into a small dashboard, using most of what the lecture covered:

- a choice of three basemaps and a layer control to switch them;
- districts shaded by **population density**, with a panel that reports the district under the pointer;
- population drawn a second way, as **circles** sized by the number of people, in their own pane above the polygons;
- map events that can be switched on and off;
- an extension that measures what SVG and canvas drawing cost.

Learning outcomes

By the end of this laboratory you will be able to:

1. Configure a map through its options, and constrain its view.
2. Offer several basemaps and overlays through a layer control, and meet the attribution each one requires.
3. Explain why a choropleth must show a rate, and show a count with proportional symbols instead.
4. Use all four hooks of `L.geoJSON`: `style`, `onEachFeature`, `filter` and `pointToLayer`.
5. Build hover highlighting with `resetStyle`, and a custom control that updates as the pointer moves.
6. Control draw order with a pane.
7. Register and remove event handlers, and explain why `off` needs a named function.
8. Measure the difference between SVG and canvas rendering.

Structure and timing

Parts 1 to 7 are the core. Parts 8 and 9 are extensions. If you run short of time, complete Part 10 first so that your work is published.

Part	What you do	Time
1	Set up the project	10 min
2	Make a point layer in QGIS	15 min
3	The page, the map and its basemaps	20 min
4	A thematic layer, done properly	25 min
5	Interaction: highlight and an info panel	25 min
6	Proportional symbols in their own pane	25 min
7	Map events, on and off	15 min
8	Filter by province *(extension)*	15 min
9	SVG or canvas *(extension)*	15 min
10	Publish and submit	15 min

What you need

Item	Requirement
Lab 2	Finished and pushed. You reuse its data and its ideas
Data	`districts.geojson` from Lab 2, with `pop_2023` and `area_km2`
QGIS	Version 3.28 or later
Visual Studio Code	With Live Server
Browser	Chrome or Edge, for the developer tools

This lab is longer on reasoning than on typing

Most of the code is short. What matters is why each piece is written the way it is: why density and not population, why a pane, why a named function. The questions at the end ask about those decisions, not about syntax.

1 Set up the project

Work in the same repository you used for Lab 2, so that Lab 2 stays published. Create a new folder for this lab inside it:

```
cd C:\webgis-lab
mkdir lab3
mkdir lab3\data
copy data\districts.geojson lab3\data\districts.geojson
code .
```

In `lab3`, create `index.html`, `style.css` and `app.js`. When you publish, this lab will appear at your Lab 2 address followed by `/lab3/`.

Open the right folder in Live Server

Live Server serves the folder VS Code has open. Open `lab3/index.html` and start Live Server from it, and check that the address bar ends in `/lab3/index.html`. The data paths in this lab are relative to that file.

Checkpoint 1

A screenshot of VS Code showing `lab3` with its three files and `data/districts.geojson`.

2 Make a point layer in QGIS

Population will be drawn twice: as a density on the polygons, and as a circle per district. Circles need a point for each district.

2.1 Point on surface, not centroid

Load `lab3/data/districts.geojson` into QGIS. Then run both of these tools on it, from **Vector**, **Geometry Tools**:

Tool	Places the point
Centroids	At the centre of mass. For a curved or hollow shape this can fall outside the polygon
Point on Surface	Guaranteed to fall inside the polygon

Find the district whose centroid lies outside its own boundary. There is one in this dataset. Zoom to it and take a screenshot showing both points.

Why it matters A symbol drawn outside its district is attributed by the reader to the neighbouring one. On a dashboard nobody checks; they read it.

2.2 Export

Export the **Point on Surface** result as GeoJSON with these settings, and record its size and feature count.

Setting	Value
File name	`lab3/data/district_points.geojson`
CRS	EPSG:4326
`COORDINATE_PRECISION`	5

The points inherit every attribute of their district, including `pop_2023`. Confirm in the attribute table that there are 160 features.

Checkpoint 2

The screenshot of the district whose centroid falls outside it, with both points visible, and the size and feature count of `district_points.geojson`.

---

3 The page, the map and its basemaps

3.1 index.html

lab3/index.html, complete

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>District Dashboard</title>

  <link rel="stylesheet"
        href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css">
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <header>
    <h1>District Dashboard</h1>
    <p class="subtitle">Population density and population of
      Pakistan's districts, 2023 census</p>
  </header>

  <div id="panel">
    <label for="province">Province</label>
    <select id="province">
      <option value="all">All provinces</option>
    </select>
    <span id="count">loading...</span>

    <label class="toggle">
      <input type="checkbox" id="coords">
      Show coordinates on click
    </label>
  </div>

  <div id="map"></div>

  <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
  <script src="app.js"></script>
```

```
</body>
</html>
```

3.2 style.css

The first half repeats Lab 2. The second half styles the two controls you will draw on the map.

lab3/style.css, complete

```
body { font-family: system-ui, sans-serif; margin: 0;
       color: #23373b; background: #f4f6f6; }
header { padding: 18px 24px 10px; }
h1 { margin: 0; font-size: 24px; }
.subtitle { margin: 4px 0 0; color: #5b6a6e; }

#panel { padding: 10px 24px 14px; display: flex; flex-wrap: wrap;
         align-items: center; gap: 12px; }
#count { color: #5b6a6e; font-size: 14px; }
.toggle { margin-left: auto; font-size: 14px; }

#map { height: calc(100vh - 150px); min-height: 420px; width: 100%; }

.info, .legend { background: white; padding: 8px 10px;
                 border-radius: 4px; font-size: 13px; line-height: 1.45;
                 box-shadow: 0 1px 4px rgba(0, 0, 0, .25); }
.info { min-width: 200px; }
.info h4 { margin: 0 0 4px; font-size: 14px; }
.info .muted { color: #6b7a7e; }

.legend i { width: 16px; height: 16px; float: left;
            margin-right: 8px; opacity: .85; }
.legend .circles { margin-top: 8px; padding-top: 6px;
                   border-top: 1px solid #e3e6e7; }
.legend svg { display: block; margin-top: 4px; }
```

3.3 The map, through its options

In Lab 2 you created the map and then called `setView`. Here every setting goes into the options object instead.

lab3/app.js, first block

```
const map = L.map('map', {
  center: [30.3753, 69.3451],
  zoom: 5,
  minZoom: 5,
  maxZoom: 14,
  maxBounds: [[21.5, 58.5], [39.0, 80.5]],
  maxBoundsViscosity: 1.0
});
```

Option	Why this value
`minZoom: 5`	The whole country fits; further out shows only neighbours
`maxZoom: 14`	District boundaries simplified to about 100 m stop looking right well before street level
`maxBounds`	Pakistan with about two degrees to spare on each side

3.4 Three basemaps and a layer control

lab3/app.js, continued

```
const osm = L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', {
  maxZoom: 19,
  attribution: '&copy; OpenStreetMap contributors'
});

const positron = L.tileLayer(
  'https://{s}.basemaps.cartocdn.com/light_all/{z}/{x}/{y}{r}.png', {
  maxZoom: 19,
  subdomains: 'abcd',
  attribution: '&copy; OpenStreetMap contributors &copy; CARTO'
});

const imagery = L.tileLayer(
  'https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y}/{x
    }', {
  maxZoom: 19,
  attribution: 'Tiles &copy; Esri'
});

positron.addTo(map);

const layerControl = L.control.layers(
  {
    'Light (Carto Positron)': positron,
    'OpenStreetMap': osm,
    'Satellite (Esri)': imagery
  },
  {},
  { collapsed: false }
).addTo(map);

L.control.scale({ imperial: false }).addTo(map);
```

The second argument to `L.control.layers` is the overlays. It is empty for now; Parts 4 and 6 add to it. Keep the reference in `layerControl`, because you need it to do that.

Read the templates closely Compare the three URL templates. Two of them end `{x}/{y}`. Esri's ends `{y}/{x}`. Leaflet fills in whatever placeholders the template contains, in whatever order they appear, so each provider's template must be copied exactly. `{s}` is replaced by one of the `subdomains` letters and `{r}` by `@2x` on high-density screens.

Why Positron is the default A thematic layer is about to be drawn on top. A busy basemap competes with it; a quiet grey one does not.

Attribution is a licence condition

Each `attribution` string is shown in the corner of the map, and it changes when you switch basemap. Do not remove it or the attribution control to tidy the map: the OpenStreetMap licence requires the credit.

Checkpoint 3

Open the Network tab, switch to each basemap in turn, and take one screenshot showing an Esri tile request with its URL visible. In your report, write down which number in that URL is the row and which is the column.

---

4 A thematic layer, done properly

4.1 Why density, not population

In Lab 2 you shaded districts by population. That map has a flaw. A large district can hold many people simply because it is large, so the shading largely repeats the districts' areas. A choropleth colours **areas**, so the value behind the colour must be one that does not grow with area: a rate, a ratio or a density.

Population is still worth showing. It is a count, and counts are shown with symbols whose size follows the count. That is Part 6.

The rule

Choropleth: densities, rates, percentages. Proportional symbols: counts. Never colour polygons by a count.

4.2 Style by density

lab3/app.js, continued

```
function show(value) {
  if (value === null || value === undefined || value === '') {
    return 'not recorded';
  }
  return typeof value === 'number' ? value.toLocaleString() : value;
}

function densityOf(p) {
  if (p.pop_2023 === null || p.pop_2023 === undefined) return null;
```

```
  return p.pop_2023 / p.area_km2;
}

function colourFor(density) {
  if (density === null) return '#c9cfd1';
  if (density > 1000) return '#08306b';
  if (density >  400) return '#2171b5';
  if (density >  150) return '#4292c6';
  if (density >   50) return '#9ecae1';
  return '#deebf7';
}

function styleFor(feature) {
  return {
    color: '#ffffff',
    weight: 1,
    fillColor: colourFor(densityOf(feature.properties)),
    fillOpacity: 0.8
  };
}
```

Density is not stored in the file. `densityOf` works it out from two attributes that are, every time a district is styled. With these breaks the 135 districts with census data fall 20, 18, 34, 41 and 22 into the five classes.

4.3 A group the layer control can hold

The layer control needs one object to switch on and off. But in Part 8 the district layer is rebuilt every time the province changes, and a control pointing at a layer that has been thrown away switches nothing. The answer is a group that stays put while its contents change.

lab3/app.js, continued

```
const districtGroup = L.featureGroup().addTo(map);
layerControl.addOverlay(districtGroup, 'Population density');
```

4.4 Load the data and draw

lab3/app.js, continued

```
let allDistricts = null;
let allPoints = null;
let districts = null;

function draw(province) {
  const keep = function (feature) {
    return province === 'all' || feature.properties.province === province;
  };

  districtGroup.clearLayers();

  districts = L.geoJSON(allDistricts, {
```

```
    filter: keep,
    style: styleFor
  });
  districtGroup.addLayer(districts);

  const n = districts.getLayers().length;
  if (n > 0) {
    map.fitBounds(districts.getBounds());
  }
  document.getElementById('count').textContent = n + ' districts';
}

async function loadJSON(path) {
  const response = await fetch(path);
  if (!response.ok) {
    throw new Error(path + ': status ' + response.status);
  }
  return response.json();
}

async function loadData() {
  try {
    [allDistricts, allPoints] = await Promise.all([
      loadJSON('data/districts.geojson'),
      loadJSON('data/district_points.geojson')
    ]);
  } catch (err) {
    console.error('Could not load the data.', err);
    document.getElementById('count').textContent = 'data failed to load';
    return;
  }
  draw('all');
}

loadData();
```

The `filter` hook In Lab 2 you filtered the array of features yourself and handed Leaflet the result. Here `L.geoJSON` does it: `filter` is called once per feature, and a feature for which it returns `false` is never drawn. `keep` reads `province` from the call to `draw` that created it, so the same function serves every province.

Two files, one wait `Promise.all` starts both requests at once and waits for both. Two `await`s in a row would fetch the second file only after the first had arrived.

4.5 A legend for the density classes

lab3/app.js, continued

```
const legend = L.control({ position: 'bottomright' });

legend.onAdd = function () {
  const div = L.DomUtil.create('div', 'legend');
  const breaks = [0, 50, 150, 400, 1000];
```

```
  const labels = ['under 50', '50 to 150', '150 to 400',
                  '400 to 1,000', 'over 1,000'];

  div.innerHTML = '<strong>People per km<sup>2</sup></strong><br>';
  for (let i = 0; i < breaks.length; i++) {
    div.innerHTML +=
      '<i style="background:' + colourFor(breaks[i] + 1) + '"></i> ' +
      labels[i] + '<br>';
  }
  div.innerHTML +=
    '<i style="background:' + colourFor(null) + '"></i> no census data';

  L.DomEvent.disableClickPropagation(div);
  return div;
};

legend.addTo(map);
```

Checkpoint 4

A screenshot of the density map with the legend, and with the layer control open showing the three basemaps and the one overlay.

---

5 Interaction: highlight and an info panel

5.1 A custom control that updates

A popup costs the user a click and covers the map. An info panel in the corner costs nothing: it reports whatever is under the pointer.

lab3/app.js, continued

```
const info = L.control({ position: 'topright' });

info.onAdd = function () {
  this._div = L.DomUtil.create('div', 'info');
  L.DomEvent.disableClickPropagation(this._div);
  L.DomEvent.disableScrollPropagation(this._div);
  this.update();
  return this._div;
};

info.update = function (p) {
  const zoom = '<div class="muted">Zoom level ' + map.getZoom() + '</div>';
  if (!p) {
    this._div.innerHTML = '<h4>District</h4>' +
      '<div class="muted">Hover over a district</div>' + zoom;
    return;
  }
```

```
  const d = densityOf(p);
  this._div.innerHTML =
    '<h4>' + p.district + '</h4>' +
    p.province + '<br>' +
    'Population: ' + show(p.pop_2023) + '<br>' +
    'Area: ' + show(p.area_km2) + ' km<sup>2</sup><br>' +
    'Density: ' + (d === null ? 'not recorded'
       : Math.round(d).toLocaleString() + ' per km<sup>2</sup>') +
    zoom;
};

info.addTo(map);
```

`update` **is yours** Leaflet calls `onAdd` once. `update` is an ordinary method you added to the control; nothing calls it until your own code does. Called with no argument it shows the resting message.

Why the two `disable` calls

The panel sits inside the map container. Without `disableClickPropagation`, clicking it also clicks the map beneath; without `disableScrollPropagation`, scrolling over it zooms the map. You will see the first one matter in Part 7.

5.2 Highlight on hover

lab3/app.js, continued

```
function highlight(e) {
  e.target.setStyle({ weight: 3, color: '#1a202c', fillOpacity: 0.95 });
  e.target.bringToFront();
  info.update(e.target.feature.properties);
}

function unhighlight(e) {
  districts.resetStyle(e.target);
  info.update();
}

function zoomToDistrict(e) {
  map.fitBounds(e.target.getBounds(), { padding: [20, 20] });
}

function bindDistrict(feature, layer) {
  layer.bindTooltip(feature.properties.district, { sticky: true });
  layer.on({
    mouseover: highlight,
    mouseout: unhighlight,
    click: zoomToDistrict
  });
}
```

Now hand `bindDistrict` to `L.geoJSON` inside `draw`:

```
  districts = L.geoJSON(allDistricts, {
    filter: keep,
    style: styleFor,
    onEachFeature: bindDistrict
  });
```

Piece	Why
`e.target`	The district that fired the event. One shared handler serves all 160
`bringToFront`	Otherwise the thick outline is half hidden under the neighbouring polygons
`resetStyle`	Re-runs `styleFor` for that district. You never store the original colour
`districts`	`resetStyle` belongs to the `L.geoJSON` layer, which is why `draw` keeps a reference to it

Checkpoint 5

A screenshot with one district highlighted, its tooltip showing, and the info panel reporting its population and density.

6 Proportional symbols in their own pane

6.1 A pane above the polygons

Polygons and circle markers are both vector paths, so by default both go into `overlayPane` and whichever is drawn later covers the other. Hovering a district calls `bringToFront`, which would lift it over the circles. A pane with a higher z-index settles the order once.

lab3/app.js, continued

```
map.createPane('symbols');
map.getPane('symbols').style.zIndex = 450;
```

`overlayPane` is 400 and `markerPane` 600, so 450 puts the circles above every polygon and below markers, tooltips and popups.

6.2 Size the circles

lab3/app.js, continued

```
function radiusFor(population) {
```

```
  return Math.sqrt(population) / 160;
}

function circleFor(feature, latlng) {
  return L.circleMarker(latlng, {
    pane: 'symbols',
    radius: radiusFor(feature.properties.pop_2023),
    color: '#7c2d12',
    weight: 1,
    fillColor: '#f97316',
    fillOpacity: 0.7
  }).bindTooltip(
    feature.properties.district + ': ' +
    feature.properties.pop_2023.toLocaleString()
  );
}

function hasPopulation(feature) {
  return feature.properties.pop_2023 !== null;
}
```

Why the square root The eye compares the **area** of a circle, and area grows with the square of the radius. If the radius followed population, a district with four times the people would look sixteen times larger. Taking the square root first makes area follow population. Lahore, at about 13 million, gets a radius of about 22 pixels.

`pointToLayer` **and** `filter` **together** Without `pointToLayer`, every GeoJSON point becomes a default pin. `hasPopulation` drops the 25 districts with no census value before anything is drawn, so `circleFor` never meets a `null`.

6.3 Draw them, and add them to the control

Below `districtGroup`, add a second group:

lab3/app.js, continued

```
const circleGroup = L.featureGroup().addTo(map);
layerControl.addOverlay(circleGroup, 'Population');
```

Then in `draw`, clear it alongside the districts, and fill it straight after `districtGroup.addLayer(districts)`:

```
  circleGroup.clearLayers();

  circleGroup.addLayer(L.geoJSON(allPoints, {
    filter: function (f) { return keep(f) && hasPopulation(f); },
    pointToLayer: circleFor
  }));
```

6.4 Add the sizes to the legend

Inside `legend.onAdd`, just before the `disableClickPropagation` line, add three reference circles drawn with the same `radiusFor` as the map, so the legend cannot disagree with it:

```
  const sizes = [10000000, 4000000, 1000000];
  const rMax = radiusFor(sizes[0]);
  const pad = 6;
  const base = pad + 2 * rMax;
  let svg = '<svg width="' + (2 * rMax + 75) +
            '" height="' + (base + 2) + '">';
  for (const s of sizes) {
    const r = radiusFor(s);
    const top = base - 2 * r;
    svg += '<circle cx="' + (rMax + 1) + '" cy="' + (base - r) +
           '" r="' + r + '" fill="#f97316" fill-opacity=".2"' +
           ' stroke="#7c2d12"/>' +
           '<line x1="' + (rMax + 1) + '" y1="' + top +
           '" x2="' + (2 * rMax + 14) + '" y2="' + top +
           '" stroke="#7c2d12" stroke-width=".6"/>' +
           '<text x="' + (2 * rMax + 17) + '" y="' + (top + 3.5) +
           '" font-size="10">' + (s / 1000000) + ' million</text>';
  }
  svg += '</svg>';
  div.innerHTML +=
    '<div class="circles"><strong>Population</strong>' + svg + '</div>';
```

6.5 Metres or pixels

Add one more overlay, to see the difference between the two circle constructors from the lecture:

lab3/app.js, continued

```
const nust = [33.6423, 72.9906];
const nustGroup = L.featureGroup([
  L.circle(nust, { radius: 25000, color: '#0f766e', weight: 2,
                   fillOpacity: 0.08 }),
  L.circleMarker(nust, { radius: 6, color: '#0f766e',
                         fillColor: '#0f766e', fillOpacity: 1 })
]).bindTooltip('NUST H-12: a 25 km circle and a 6 pixel circleMarker');
layerControl.addOverlay(nustGroup, 'NUST, 25 km');
```

Turn it on in the layer control and zoom from level 5 to level 10. Note what happens to each circle on screen, and measure the 25 km circle against the scale bar.

Checkpoint 6

Two screenshots: the full dashboard with circles above the density shading and the legend showing both parts; and the NUST overlay at two different zoom levels.

---

7 Map events, on and off

7.1 A handler you can remove

The checkbox in the panel switches a click handler on and off. While it is on, clicking the map shows the coordinates of that point.

lab3/app.js, continued

```
function showCoordinates(e) {
  L.popup()
    .setLatLng(e.latlng)
    .setContent(e.latlng.lat.toFixed(4) + ', ' + e.latlng.lng.toFixed(4))
    .openOn(map);
}

document.getElementById('coords')
  .addEventListener('change', function (event) {
    if (event.target.checked) {
      map.on('click', showCoordinates);
    } else {
      map.off('click', showCoordinates);
    }
  });
```

Why `showCoordinates` has a name

`map.off` removes a handler by finding the very function you passed to `map.on`. Suppose you had written this instead:

```
map.on('click', function (e) { ... });
```

That function has no name and no variable holding it, so nothing you could pass to `off` would match it. The handler could never be removed.

Two listeners, two systems `addEventListener` is the browser's own event system, used on page elements. `map.on` is Leaflet's, used on maps and layers. They look alike and do the same job for different objects.

7.2 Keep the panel's zoom level current

lab3/app.js, continued

```
map.on('zoomend', function () {
  info.update();
});
```

`zoomend` fires once when a zoom finishes. `zoom` would fire continuously during the animation.

7.3 Try these

1. Tick the checkbox and click the sea south of Karachi. Then click a district. Both handlers run: write down what you see.
2. With the checkbox ticked, click the legend. No popup appears. In the legend's `onAdd`, remove the line that calls `disableClickPropagation`, reload, and click the legend again. Put the line back.
3. Untick the checkbox and confirm clicks on the sea do nothing.

Checkpoint 7

A screenshot of a coordinate popup on the sea, and your notes from the three steps above.

---

8 Filter by province

Extension exercise

`draw` already takes a province and filters by it. All that is missing is the drop-down.

lab3/app.js, continued

```
function fillProvinceList() {
  const select = document.getElementById('province');
  const provinces = [...new Set(
    allDistricts.features.map(f => f.properties.province)
  )].sort();
  for (const name of provinces) {
    const option = document.createElement('option');
    option.value = name;
    option.textContent = name;
    select.appendChild(option);
  }
}

document.getElementById('province')
  .addEventListener('change', function (event) {
    draw(event.target.value);
  });
```

Call `fillProvinceList()` in `loadData`, just before `draw('all')`.

What to check Select Gilgit Baltistan. The count reads 14 districts but no circles appear. Explain why in your report, and say whether the dashboard should tell the user so.

Checkpoint 8

A screenshot with one province selected and the layer control open, showing that the overlays still switch the rebuilt layers on and off.

---

9 SVG or canvas

Extension exercise

9.1 Make the renderer switchable

Add this as the very first line of `app.js`:

```
const useCanvas = new URLSearchParams(window.location.search).has('canvas');
```

Add `preferCanvas: useCanvas` to the options of `L.map`. Opening `index.html?canvas` now draws every vector onto a canvas, and `index.html` on its own still uses SVG.

9.2 Time the drawing

Make the first line of `draw`

```
  const t0 = performance.now();
```

and the last

```
  console.log('draw(' + province + '): ' + n + ' districts in ' +
              (performance.now() - t0).toFixed(1) + ' ms, ' +
              (useCanvas ? 'canvas' : 'SVG'));
```

9.3 Measure

For each renderer, reload five times and record the time printed for `draw(all)`. Then, in the Elements tab, find `leaflet-overlay-pane` and count what it contains.

	SVG	Canvas
`draw(all)`, median of five, ms		
Elements inside `leaflet-overlay-pane`		
Hover highlight still works?		
Can you inspect one district in Elements?		

Reading the result With 160 districts and 135 circles, the difference is small and may even favour SVG. The lecture's guidance is that canvas starts to pay at a few thousand features. Say in your report what you would expect for the tehsil layer (administrative level 3), which has several times as many features, each with more vertices, and why.

Checkpoint 9

The completed table, and screenshots of the Elements tab in both modes.

---

10 Publish and submit

10.1 Commit and push

```
cd C:\webgis-lab
git add -A
git commit -m "Lab 3: district dashboard"
git push
```

Wait for GitHub Pages to rebuild, then open `https://<your-username>.github.io/webgis-lab-<roll>/lab3/` on your phone. Check that Lab 2 still works at its old address.

10.2 Write the report

Create `lab3/report.md` containing:

1. Your checkpoint screenshots.
2. The size and feature count of `district_points.geojson`, and the name of the district whose centroid falls outside it.
3. The row and column in your Esri tile URL.
4. Your notes from the three steps in Part 7.
5. The table from Part 9, if you completed it.
6. Your answers to the questions below.

10.3 Questions

1. Lab 2 shaded districts by population. Name one large, thinly populated district and one small, crowded one, and explain how each would mislead on the Lab 2 map.
2. `radiusFor` takes a square root. What would the map suggest about Lahore against a district of 3 million if it did not?
3. Why does the layer control hold `districtGroup` rather than the `L.geoJSON` layer itself?

4. What would go wrong if the circles were drawn in `overlayPane` instead of the `symbols` pane? Describe what the user would see when hovering.

5. Why can `map.off` remove `showCoordinates`, when it could not remove an anonymous function passed to `map.on`?

6. On the NUST overlay, which circle would you use to show a 25 km evacuation zone, and which to mark the campus on a national map? Why?

10.4 Marking

Item	Marks
Published dashboard with basemaps, density shading and legend	20
Hover highlight, info panel and click to zoom	15
Proportional symbols in their own pane, with legend	15
Coordinate click handler that switches on and off	10
Point layer, with the centroid comparison	10
Answers to the six questions	25
Checkpoint screenshots and commit history	5
Extensions in Parts 8 and 9	+10 bonus

Submit two links on LMS: your repository and your live `lab3` page.

Appendix A: Troubleshooting

Symptom	Cause and fix
`data failed to load`, 404 in the console	Live Server was started from the wrong folder, or the file is not in `lab3/data`. Check the address bar ends in `/lab3/index.html`
Satellite basemap shows grey or the wrong place	The `{y}/{x}` order in the Esri template was changed. Copy it exactly
Highlight never goes away	`resetStyle` was called on something other than the `L.geoJSON` layer, or `districts` was never assigned in `draw`
`Cannot read properties of null (reading 'resetStyle')`	The same: `draw` must assign `districts = L.geoJSON(...)`
Circles hidden under polygons	`pane: 'symbols'` is missing from the circle options, or the pane was created after the circles were drawn
`Cannot read properties of null (reading 'toLocaleString')`	A district with no population reached `circleFor`. Add the `hasPopulation` filter
Layer control switches nothing after changing province	The control was given the `L.geoJSON` layer instead of the group
Clicking the legend opens a coordinate popup	`disableClickPropagation` is missing from that control
Coordinate popups cannot be turned off	The handler passed to `map.on` is an anonymous function. Name it
Info panel zoom level lags behind	The `zoomend` handler is missing

Appendix B: Reference

The four hooks of `L.geoJSON`, as used here

Hook	In this lab
`filter`	`keep`: one province, and `hasPopulation` for the circles
`style`	`styleFor`: colour by density
`onEachFeature`	`bindDistrict`: tooltip and three handlers
`pointToLayer`	`circleFor`: a circle marker instead of a pin

Default panes

Pane	z-index	Holds
`tilePane`	200	Basemap tiles
`overlayPane`	400	Vector paths: polygons, lines, circles
`symbols`	450	This lab's population circles
`shadowPane`	500	Marker shadows
`markerPane`	600	Marker icons
`tooltipPane`	650	Tooltips
`popupPane`	700	Popups

Optional, if you finish everything

1. Add Leaflet.markercluster and draw the 160 points as clustered markers on a separate overlay.
2. Show the density value as a permanent tooltip on each district above zoom level 8, using `zoomend` to add and remove them.
3. Save the map's centre and zoom in `localStorage` on `moveend`, and restore them when the page loads.
4. Replace the fixed density breaks with quantiles computed from the data, and update the legend to match.